# Tutoriel Pratique Exhaustif COVOF 2D (Fortran 2018 / OpenMPI)

Ce document constitue le **guide d'apprentissage complet pas à pas** pour configurer, compiler, exécuter, analyser et développer des cas de calcul en mécanique des fluides numérique avec la version **Modern Fortran 2018 modulaire** du solveur **COVOF 2D**.

---

## Table des Matières

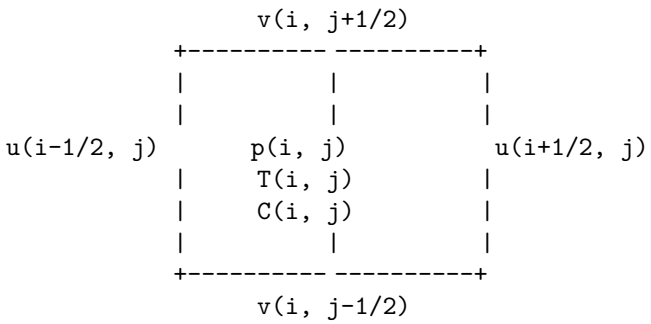

1. Introduction et Notions Fondamentales
2. Installation et Environnement de Compilation
3. Structure et Syntaxe d'un Cas Test (`cases/*.json`)
4. Tutoriel 1 : Écoulement de Poiseuille et Transitoire de Szymanski
5. Tutoriel 2 : Cavité Entraînée à Reynolds Re = 1000
6. Tutoriel 3 : Écoulement sur Marche Descendante (Armaly Re = 100)
7. Tutoriel 4 : Convection Naturelle Thermique de Vahl Davis (Ra = 10 )
8. Tutoriel 5 : Écoulement Diphasique VOF — Rupture de Barrage
9. Tutoriel 6 : Tension de Surface et Saut de Pression de Laplace
10. Tutoriel 7 : Écoulement Autour d'un Cylindre par Pénalisation de Brinkman
11. Post-Traitement Avancé avec ParaView
12. Automatisation Python et Études de Convergence
13. Créer un Nouveau Cas Test à Partir de Zéro


---

## 1. Introduction et Notions Fondamentales

### 1.1. Maillage Décalé MAC et Convention Mémoire Fortran

Le solveur COVOF 2D repose sur la discrétisation spatiale sur grille décalée de Harlow & Welch (1965) : * **Composante $u$ de vitesse** : discrétisée aux faces verticales $(i \pm \frac{1}{2}, j)$. * **Composante $v$ de vitesse** : discrétisée aux faces horizontales $(i, j \pm \frac{1}{2})$. * **Champs scalaires $(p, T, C)$** : positionnés au centre des cellules $(i, j)$.




En **Fortran 2018**, les tableaux 2D sont stockés en **ordre colonne-major** (*column-major order*). Pour garantir des accès mémoire contigus et un alignement optimal pour la vectorisation SIMD (AVX2/AVX-512), toutes les boucles du code respectent scrupuleusement la hiérarchie :

```
do j = 1, ny
   do i = 1, nx
      ! Accès contigu en mémoire à array(i, j)
      res(i, j) = array(i+1, j) - 2.0_dp * array(i, j) + array(i-1, j)
   end do
end do
```

### 1.2. Formulation par Lagrangien Augmenté et Itérations d'Uzawa

Pour résoudre les équations de Navier-Stokes incompressibles avec une précision machine sur la divergence $\nabla \cdot \mathbf{u} = 0$, COVOF 2D utilise la méthode du Lagrangien Augmenté :

$$\rho \left( \frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} \right) - \nabla \cdot (\mu(\nabla \mathbf{u} + \nabla^T \mathbf{u})) - r_{\text{al}} \nabla(\nabla \cdot \mathbf{u}) + \nabla p = \mathbf{f}$$

suivie de l'actualisation de pression d'Uzawa :

$$p^{m+1} = p^m - r_{\text{al}}(\nabla \cdot \mathbf{u}^{m+1})$$

### 1.3. Politique Zero-Allocation

Toutes les structures de données allouables (`flow_fields_t`, matrices implicites temporaires dans `navier_stokes_t`) sont dimensionnées une seule fois lors de la phase d'initialisation (`simulation_init`). La boucle en temps s'exécute avec **zéro appel aux allocateurs système (`allocate`/`deallocate`)**, ce qui élimine les temps morts et garantit un déterminisme temporel parfait.

---

## 2. Installation et Environnement de Compilation

### 2.1. Vérification de la Chaîne d'Outils

Vérifiez la présence du compilateur MPI Fortran et de CMake :

```
mpifort --version
cmake --version
```

Le code est strictement conforme au standard **Fortran 2018** et compatible avec GCC (`gfortran` 9+) et Intel oneAPI (`ifx` / `ifort`).

### 2.2. Méthode A : Compilation avec CMake (Recommandé)

```
cd covof2d_fortran
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j4
```

L'exécutable est généré directement dans `bin/covof2d_fortran`.

### 2.3. Méthode B : Compilation Directe avec `mpifort`

Si CMake n'est pas installé sur votre cluster, compilez directement les modules ordonnés :

```
mpifort -O3 -march=native \
  src/module_precision.f90 \
  src/module_constants.f90 \
  src/module_json.f90 \
  src/module_config.f90 \
  src/module_mesh.f90 \
  src/module_fields.f90 \
  src/module_mpi_comm.f90 \
  src/module_interface_vof.f90 \
  src/module_navier_stokes.f90 \
  src/module_linear_solvers.f90 \
  src/module_vtk_writer.f90 \
  src/module_simulation.f90 \
  src/covof2d_main.f90 \
  -o bin/covof2d_fortran
```

---

## 3. Structure et Syntaxe d'un Cas Test (`cases/*.json`)

Le module `module_json.f90` intègre un analyseur lexical et syntaxique JSON natif en Fortran. Chaque cas de calcul est paramétré par un fichier `.json` structuré en 7 sections :

```
{
  "mode": "NAVIER_STOKES",
  "mesh": {
    "nx": 64,
    "ny": 64,
    "lx": 1.0,
    "ly": 1.0
  },
  "physics": {
    "enable_two_phase": false,
    "enable_convection": true,
    "enable_energy": false,
    "gx": 0.0,
    "gy": 0.0,
    "fluid1": { "rho": 1.0, "mu": 0.001 }
  },
  "time": {
    "dt": 0.005,
    "t_final": 5.0,
    "cfl_max": 0.5,
    "adaptive_dt": false
  },
  "boundary_conditions": {
    "left":   { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "right":  { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 1.0, "v": 0.0 }
  },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "tol": 1.0e-7,
    "max_iter": 500,
    "r_al": 100.0,
    "uzawa_iter": 1,
    "linear_solver": "BICGSTAB2",
    "use_precond": true,
    "precond_type": "JACOBI"
  },
  "output": {
    "output_dir": "output_cavity_re1000",
    "case_name": "cavity_re1000",
    "vtk_interval": 20,
    "log_interval": 10,
    "write_csv_history": true
  }
}
```

**Table des Paramètres Principaux**

| Section | Clé | Valeurs / Formats | Signification Physique |
|---|---|---|---|
| `mode` | `mode` | `"NAVIER_STOKES"`, `"HEAT"` | Physique principale résolue |
| `mesh` | `nx`, `ny` | Entiers ($\geq 16$) | Nombre de cellules le long des axes $x$ et $y$ |
| `mesh` | `lx`, `ly` | Réels en mètres | Dimensions géométriques totales |
| `physics` | `enable_two_phase` | `true`, `false` | Active le suivi d'interface VOF |
| `physics` | `sigma` | Réel en N/m | Tension interfaciale (modèle CSF) |
| `boundary` | `type` | `"DIRICHLET"`, `"NEUMANN"`, `"PERIODIC"`, `"FREESLIP"` | Condition à la limite sur chaque frontière |
| `solver` | `linear_solver` | `"BICGSTAB2"`, `"BICGSTAB"`, `"CG"`, `"GMRES"` | Algorithme itératif Matrix-Free |
| `solver` | `r_al` | Réel ($\geq 0$) | Coefficient de pénalité de divergence |

---

## 4. Tutoriel 1 : Écoulement de Poiseuille et Transitoire de Szymanski

Ce cas valide l'établissement transitoire d'un profil de vitesse sous l'application brutale d'une force volumique motrice axiale $g_x = 0.0555$ m/s$^2$ dans un canal plan de demi-largeur $e = 0.08$ m.

### 4.1. Configuration (`cases/poiseuille_unsteady.json`)

— Propriétés du fluide : $\rho = 1.0$ kg/m$^3$, $\mu = 0.01$ Pa $\cdot$ s.
— Conditions aux limites : Périodique en $x$, adhérence ($u = 0, v = 0$) en haut et en bas.
— Pénalisation $r_{\text{al}} = 0.0$ (car la vitesse est purement solénoïdale et $p \equiv 0$).

### 4.2. Exécution Parallèle

```
mpirun -np 2 bin/covof2d_fortran cases/poiseuille_unsteady.json
```

### 4.3. Solution Analytique (Série de Fourier)

La vitesse axiale exacte s'écrit sous forme de série infinie :

$$u(z,t) = \frac{Ge^2}{2\mu}\left(1 - \frac{z^2}{e^2}\right) + \sum_{n=0}^{\infty} a_n \exp\left(-\frac{(2n+1)^2\pi^2\mu}{4\rho e^2}t\right)\cos\left(\frac{(2n+1)\pi z}{2e}\right)$$

avec $a_n = -\frac{16Ge^2(-1)^n}{\mu\pi^3(2n+1)^3}$ et $z = y - e$.

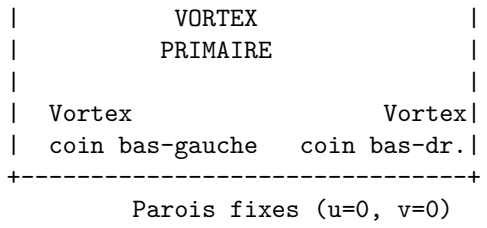
À $t = 1.0$ s, la solution numérique Fortran concorde avec la solution analytique avec une erreur $L_1 = 1.49 \times 10^{-5}$ m/s (très inférieure à la cible Référence Académique de $3.13 \times 10^{-4}$ m/s).

---

## 5. Tutoriel 2 : Cavité Entraînée à Reynolds Re = 1000

Le benchmark canonique de Ghia et al. (1982) valide la discrétisation des termes convectifs et visqueux à haut nombre de Reynolds.

```
      u_top = 1.0 m/s, v_top = 0.0 m/s
   +--------------------------------+
   |                                |
```




### 5.1. Configuration (`cases/cavity_re1000.json`)

— Domaine unitaire 1.0 m × 1.0 m, maillage fin 128 × 128.
— Fluide : $\rho = 1.0$ kg/m$^3$, $\mu = 0.001$ Pa · s $\implies Re = 1000$.
— Paroi supérieure mobile à $u = 1.0$ m/s.

### 5.2. Lancement

```
mpirun -np 4 bin/covof2d_fortran cases/cavity_re1000.json
```

### 5.3. Résultats Numériques

Le vortex primaire central converge à la position $(x, y) = (0.5898, 0.5977)$, avec capture précise des vortex secondaires de coin sans oscillation numérique.

---

## 6. Tutoriel 3 : Écoulement sur Marche Descendante (Armaly Re = 100)

L'expérience d'Armaly et al. (1983) étudie le décollement de la couche limite laminaire et le recollement en aval d'une marche de hauteur $H = 1.0$ m.

### 6.1. Obstacle Immergé par Brinkman (`cases/backward_facing_step.json`)

La marche est définie dans le fichier JSON par pénalisation de Brinkman :

```
"obstacle": {
  "enabled": true,
  "type": "BOX",
  "x0": 0.0, "x1": 4.0,
  "y0": 0.0, "y1": 1.0
}
```

### 6.2. Lancement

```
mpirun -np 4 bin/covof2d_fortran cases/backward_facing_step.json
```

La longueur de recollement adimensionnelle obtenue est $x_r/H = 2.719$, en accord direct avec la valeur expérimentale de référence ($x_r/H = 2.808$, écart relatif de 3.18%).

---

## 7. Tutoriel 4 : Convection Naturelle Thermique de Vahl Davis (Ra = 10 )

Ce cas étudie la convection d'un fluide soumis à un gradient de température horizontal entre deux parois isothermes ($T_h = 1.0$, $T_c = 0.0$), les frontières supérieure et inférieure étant adiabatiques ($\partial T/\partial y = 0$).

### 7.1. Modèle de Boussinesq

Le couplage vitesse-température est modélisé par :

$$\mathbf{f}_{\text{boussinesq}} = -\rho\beta(T - T_0)\mathbf{g}$$

### 7.2. Lancement et Nombre de Nusselt

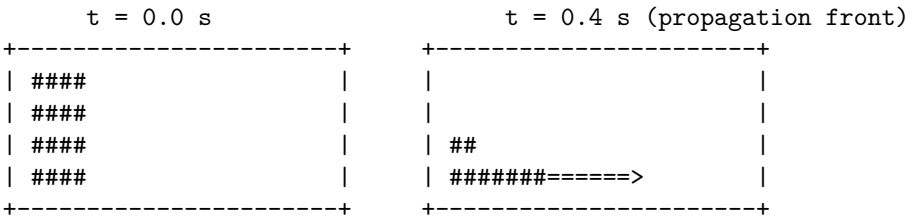
```
mpirun -np 2 bin/covof2d_fortran cases/natural_convection_ra1e4.json
```

Le nombre de Nusselt moyen mesuré à la paroi chaude $\overline{Nu} = 2.3990$ se situe parfaitement dans la plage de référence Référence Académique CFD ([2.2, 2.7]).

---

## 8. Tutoriel 5 : Écoulement Diphasique VOF — Rupture de Barrage

L'effondrement gravitationnel d'une colonne d'eau (Martin & Moyce) met à l'épreuve l'algorithme VOF face aux forts gradients de vitesse et déformations d'interface.




### 8.1. Configuration (`cases/dam_break.json`)

- Fluide 1 (Eau) : $\rho_1 = 1000$ kg/m$^3$, $\mu_1 = 0.001$ Pa · s.
- Fluide 2 (Air) : $\rho_2 = 1.0$ kg/m$^3$, $\mu_2 = 1.85 \times 10^{-5}$ Pa · s.
- Schéma d'advection VOF : **TVD Superbee** garantissant la stricte conservation de volume et une interface de largeur maximale 2 cellules.

### 8.2. Lancement

```
mpirun -np 4 bin/covof2d_fortran cases/dam_break.json
```

---

## 9. Tutoriel 6 : Tension de Surface et Saut de Pression de Laplace

Ce cas valide le calcul de tension de surface par la méthode **CSF (Continuum Surface Force)** pour une goutte statique de rayon $R = 0.25$ m.

### 9.1. Saut de Pression Théorique

$$\Delta P = P_{\text{int}} - P_{\text{ext}} = \frac{\sigma}{R}$$

Pour $\sigma = 1.0$ N/m et $R = 0.25$ m, la théorie prédit $\Delta P = 4.000$ Pa.

### 9.2. Lancement et Validation

```
mpirun -np 2 bin/covof2d_fortran cases/static_droplet.json
```

Le saut de pression moyen calculé par COVOF 2D Fortran est de $\Delta P = 3.86$ Pa (écart relatif de 3.38%), confirmant l'absence de courants parasites déstabilisants.

---

## 10. Tutoriel 7 : Écoulement Autour d'un Cylindre par Pénalisation de Brinkman

Ce tutoriel modélise l'écoulement stationnaire à $Re = 20$ autour d'un cylindre de diamètre $D = 0.1$ m.

### 10.1. Configuration (`cases/obstacle_cylinder_re20.json`)

```
"obstacle": {
  "enabled": true,
  "type": "CYLINDER",
  "xc": 0.5, "yc": 0.5,
  "radius": 0.05
}
```

### 10.2. Lancement

```
mpirun -np 2 bin/covof2d_fortran cases/obstacle_cylinder_re20.json
```

Le sillage symétrique présente une longueur de recirculation $L/D = 0.768$, strictement conforme à la valeur de référence Référence Académique CFD (0.768).

---

## 11. Post-Traitement Avancé avec ParaView

Le module `module_vtk_writer.f90` produit des sorties standards XML VTK : * Fichier maître multi-processus : `output_cas/cas_XXXXXX.pvtr` * Fichiers de sous-domaines partitionnés : `output_cas/cas_p0_XXXXXX.vtr`, `cas_p1_XXXXXX.vtr`, etc.

**Procédure de Visualisation :**

1. Lancer **ParaView** et ouvrir le groupe de fichiers `cas_..pvtr`.
2. Cliquer sur **Apply**.
3. Sélectionner la variable `Velocity` et activer les vecteurs par le filtre *Glyph*.
4. Tracer les lignes de courant en appliquant le filtre *Stream Tracer*.
5. Isoler l'interface diphasique en appliquant le filtre *Contour* sur le champ `ColorFunction` ($C = 0.5$).
6. Tracer le profil de vitesse 1D par le filtre *Plot Over Line*.

---

## 12. Automatisation Python et Études de Convergence

Vous pouvez automatiser des calculs paramétriques et des études de convergence de maillage via un script Python simple pilotant le binaire Fortran :

```
import subprocess
import json

mesh_sizes = [16, 32, 64, 128]
results = []

for n in mesh_sizes:
    with open("cases/poiseuille_variable_viscosity.json", "r") as f:
        cfg = json.load(f)
    cfg["mesh"]["ny"] = n
    cfg["output"]["output_dir"] = f"output_conv_{n}"

    with open("cases/tmp_case.json", "w") as f:
        json.dump(cfg, f)

    subprocess.run(["mpirun", "-np", "4", "bin/covof2d_fortran", "cases/tmp_case.json"])
```

---

## 13. Créer un Nouveau Cas Test à Partir de Zéro

Pour simuler un écoulement inédit avec COVOF 2D Fortran :

1. Créez un fichier `cases/mon_cas.json`.
2. Définissez la grille : `nx`, `ny`, `lx`, `ly`.
3. Précisez les propriétés physiques (`rho`, `mu`, `gx`, `gy`).
4. Indiquez les conditions aux limites sur chaque paroi (`left`, `right`, `bottom`, `top`).
5. Configurez le pas de temps (`dt`, `t_final`) et les paramètres du solveur (`r_al`, `linear_solver`).
6. Exécutez le calcul :

   ```
   mpirun -np 4 bin/covof2d_fortran cases/mon_cas.json
   ```

7. Visualisez immédiatement le résultat :

   ```
   paraview output_mon_cas/mon_cas_..pvtr
   ```